
Table of Contents

PREAMBLE	1
=====	=====
1. COMMENTING	1
=====	=====
=====	=====
2. AUDIO PLOTTING	5
=====	=====
=====	=====
3. AUDIO CUSTOMIZATION	8
=====	=====
=====	=====
4. AI for Audio Enhancement	10
=====	=====
=====	=====
1. Time vector	16
2. Fundamental frequency (same as key_to_note_violin)	16
3. Harmonic synthesis	16
4. Normalise to unit peak before scaling by A	17
5. Attack envelope (very short, 5 ms exponential)	17
6. Apply envelope, scale by A, and return real part	17

PREAMBLE

DO NOT REMOVE THE LINE BELOW

```
clear; close all;
```

```
=====
=====
```

1. COMMENTING

```
=====
=====
```

MAKE SURE FILE BELOW IS IN SAME DIRECTORY AS THIS FILE

```
type('eel3135_lab02_comment.m')
```

```
% ACKNOWLEDGEMENTS / REFERENCES:
```

```
% This code uses functions written by Ken Schutte in 2019, which is used to
% read and decode midi files. The code is under a GNU General
% Public License, enabling us to run, study, share, and modify the
% software.
```

```

%
% More info can be found at: http://www.kenschutte.com/midi

%% INITIAL SETUP
clear
close all
clc

%% DEFINE MUSIC

% INITIAL VARIABLES
Fs = 44100; % Sampling frequency in Hz (samples/second) for audio
synthesis/playback

% Read the MIDI file and decode it into a matrix of note events like pitch
% and timing
[Notes, endtime] = midiInfo(readmidi('gym.mid'), 0, 2);
L = size(Notes,1); % Number of note events (rows) in the Notes matrix. 97
in gym.mid for example

% Build the song by synthesizing one note per MIDI event using:
% amplitude = 1, key numbers from Notes(:,3), and durations = end-start times
x = build_song(ones(L,1), Notes(:,3), Notes(:,6)-Notes(:,5), Fs);

% Compute the total number of samples needed for the full song duration,
% using total time = sum(durations) and converting to samples with Fs
tot_samples = ceil(sum(Notes(:,6)-Notes(:,5))*Fs);

% Create a time vector in seconds for plotting x, then plot x.
% The first subplot shows the entire waveform; the second zooms into the
first 0.1 seconds.
t = 0:1/Fs:(tot_samples-1)/Fs; % Time axis (seconds) corresponding to each
sample in x.
figure(1); % ^ Start at 0, sample at integer intervals
of the sampling period,
subplot(211) % ^ stop at the (# of samples) * (duration of
each sample) to get the end.
plot(t, x); % '^' indicates that these are a continuation
of the previous comment
xlabel('Time [s]')
ylabel('Amplitude')
subplot(212)
plot(t, x);
xlabel('Time [s]')
ylabel('Amplitude')
axis([0 0.1 -1 1]) % Zoom: show first 0.1 s and amplitude range [-1, 1]

% Normalize x to prevent clipping (scale into [-1, 1]) and save as a WAV
file at Fs
audiowrite('example_music.wav', x./max(abs(x)), Fs);

%scales the values of audio signal x to fit in the range [-1,1], and then
%sends the data to the speaker at the sampling frequency.

```

```

input('Click any button to play sound')
soundsc(x, Fs);

% =====
% YOU DO *NOT* NEED TO DESCRIBE THESE LINES (your free to figure it out
though)
W = 0.1;      % Window size
tic;
for mm = 1:ceil(tot_samples/Fs/W)
    figure(1)
    % PAUSE UNTIL NEXT FRAME
    xlim([(mm-1)*W+[0 W]]); % Set limits of plot
    tm = toc;                % Check current time
    if mm*W < tm, disp(['Warning: Visualization is ' num2str(mm*W-tm) 's
behind']); end
    drawnow; pause(mm*0.1-tm); % Synchronize with clock
end
% =====

%%
% =====
% SUPPORTING FUNCTIONS FOUND BELOW
% Add comments appropriately below
% =====

function x = key_to_note(A, key, dur, fs)
% key_to_note: Generate a sampled sinusoidal note from amplitude A, piano
key number,
% duration dur (seconds), and sampling frequency fs (Hz).
%
% Input Args:
%   A: complex amplitude
%   key: number of the note on piano keyboard
%   dur: duration of each note (in seconds)
%   fs: A scalar sampling rate value
%
% Output:
%   x: sinusoidal waveform of the note

% Convert duration into an integer number of samples,
% create a time vector, and convert the key number into frequency
N = floortol(dur*fs); % Number of samples in this note
t = (0:(N-1)).'/fs;   % Column time vector from 0 to (N-1) * Ts
freq = (440/32)*2^((key-9)/12); % Key number -> frequency

% Generate a real sinusoid with complex amplitude A at frequency "freq"
% 2*pi*freq = angular frequency
x = real(A*exp(1j*2*pi*freq*t));

```

end

```
function x = build_song(As, keys, durs, fs)
% build_song: Create a full audio signal by generating notes from arrays of
% amplitudes,
% key numbers, and durations, and placing each note into the correct time
% region.
%
% Input Args:
%   As: A length-N array of complex amplitudes for building notes
%   keys: A length-N array of key numbers (which key on a keyboard) for
% building notes
%   durs: A length-N array of durations (in seconds) for building notes
%   fs: A scalar sampling rate value
%
% Output Args:
%   x: A length-(N*fs) length raw audio signal
%
% Pre-allocate an output vector long enough to hold the entire song
x = zeros(ceil(sum(durs)*fs), 1);
for k = 1:length(keys)

    % Generate the k-th note, and compute the start time of this note
    % as the sum of all previous durations
    note = key_to_note(As(k), keys(k), durs(k), fs);
    start_time = sum(durs(1:k-1));

    % Convert the note's start time and duration (seconds) into sample
indices,
    % then add the note samples into the correct segment of the full
song x
    n1 = floortol(start_time*fs) + 1;
    n2 = floortol(start_time*fs) + floortol(durs(k)*fs);
    x(n1:n2) = x(n1:n2) + note;

end
```

end

```
function x = floortol(x)
%FLOORTOL Apply floor operation after adding 0.5 to ensure no
% floating-point rounding errors that unintendedly decrease the
% value
%
    x = floor(x+0.5);

end
```

2. AUDIO PLOTTING

```
fs = 8000;

% -----
% 2 (a)
% -----

t = 0:1/fs:0.02;

s1 = 2*cos(500*pi*t - 3*pi/4);
s2 = 2*cos(200*pi*t - pi/4);
s3 = 3*cos(600*pi*t + pi/3);

figure;

subplot(3,1,1)
plot(t, s1);
xlabel('Time (seconds)');
ylabel('Amplitude');
title('s_1(t)');

subplot(3,1,2)
plot(t, s2);
xlabel('Time (seconds)');
ylabel('Amplitude');
title('s_2(t)');

subplot(3,1,3)
plot(t, s3);
xlabel('Time (seconds)');
ylabel('Amplitude');
title('s_3(t)');

% -----
% 2 (b)
% -----

% Fundamental period calculation:
% f1 = 250 Hz, f2 = 100 Hz, f3 = 300 Hz
% f0 = gcd(100,250,300) = 50 Hz
```

```

% T0 = 1/50 = 0.02 s
% Plot first 4*T0 = 0.08 s

t = 0:1/fs:0.08;

s1 = 2*cos(500*pi*t - 3*pi/4);
s2 = 2*cos(200*pi*t - pi/4);
s3 = 3*cos(600*pi*t + pi/3);

x = s1 + s2 + s3; %add plots

length(t)
length(x) %make sure lengths match

figure;
plot(t, x);
xlabel('Time (seconds)');
ylabel('Amplitude');
title('x(t) = s_1(t) + s_2(t) + s_3(t) (First Four Fundamental Periods)');

% -----
% 2(c)
% -----
fs = 8000;
T = 5; % length of signal in seconds

N = fs*T; % 40000 samples exactly
t = (0:N-1)/fs; % time vector for 5 seconds

s1 = 2*cos(500*pi*t - 3*pi/4);
s2 = 2*cos(200*pi*t - pi/4);
s3 = 3*cos(600*pi*t + pi/3);

x = s1 + s2 + s3;

% Normalize to prevent clipping and save as WAV
x_norm = x ./ max(abs(x));
audiowrite('x.wav', x_norm, fs);

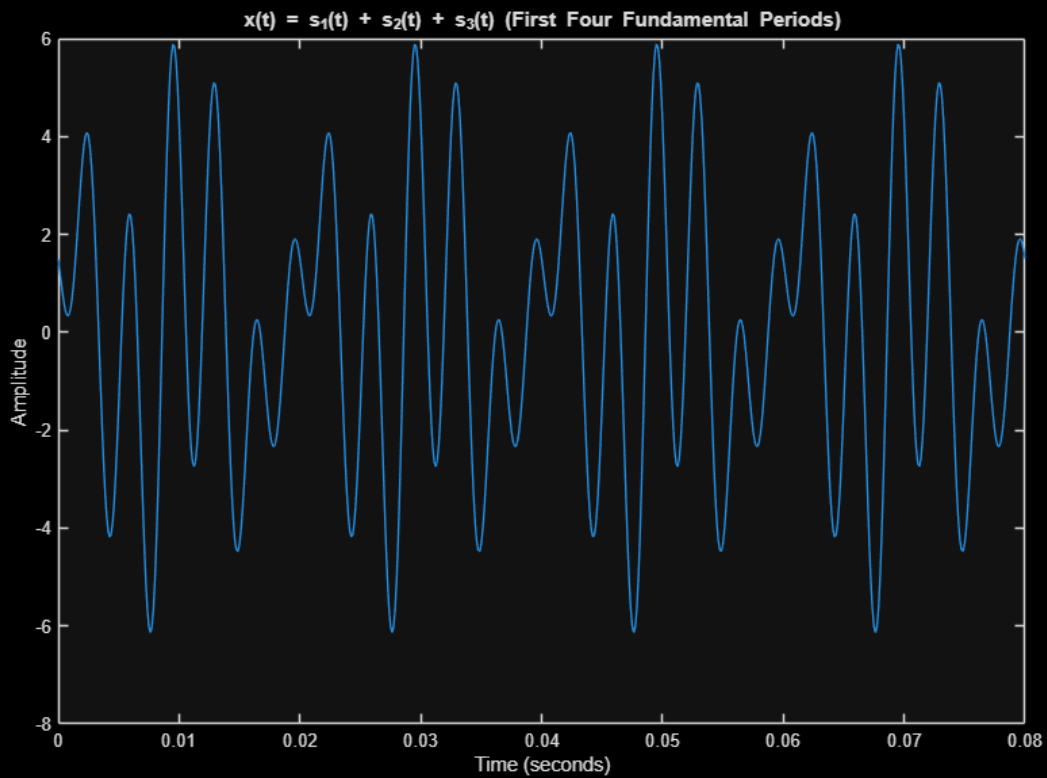
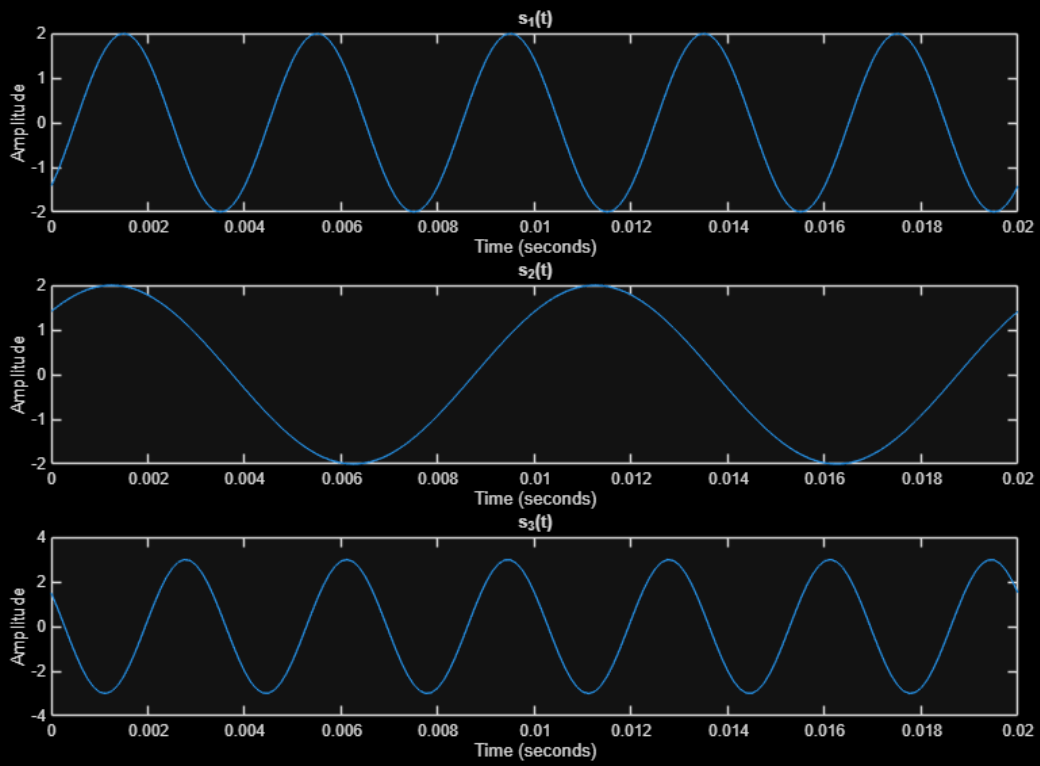
ans =

    641

ans =

    641

```



3. AUDIO CUSTOMIZATION

```
% DEFINE MUSIC
% Music is "Terra's Theme" from the Final Fantasy 6 Soundtrack
fs      = 32000;
As      = [87    101    105    108    87    88    90    102    78    108    102
103    108    86    90    88    104    110    87    99    106    105    0];
keys    = [79    81    82    86    82    81    79    81    74    79    81
82    86    82    81    79    81    86    82    84    86    89    0];
durs    = [2     2     2     2    25     2     2     8    21     2     2
2     2    25     2     2     8    21     2     2     2     2    20]/10;
As = As.^4; As = As./max(abs(As)); % Add a little bit of "punch"
ratios = As/(mean(As)*2);

% -----
% 3(a)
% -----
music_sine = build_song(As, keys, durs, fs); % generate sine-
song
music_sine = music_sine ./ max(abs(music_sine)); % normalize to
avoid clipping
audiowrite('music_sine.wav', music_sine, fs); % save as WAV

% -----
% 3(b)
% -----
% Only need to modify function -- this area can be empty

% -----
% 3(c)
% -----
% Only need to modify function -- this area can be empty

% -----
% 3(d)
% -----
music_violin = build_song_violin(As, keys, durs, fs); % generate violin
song
music_violin = music_violin ./ max(abs(music_violin)); % normalize
```

```

audiowrite('music_violin.wav', music_violin, fs);           % save WAV

% -----
% 3(e)
% -----
% Only need to modify function -- this area can be empty

% -----
% 3(f)
% -----
% Only need to modify function -- this area can be empty

% -----
% 3(g)
% -----

music_adsr = build_song_adsr(As, keys, durs, ratios, fs);
music_adsr = music_adsr ./ max(abs(music_adsr));
audiowrite('music_adsr.wav', music_adsr, fs);

% -----
% 3(h)
% -----
% Build full signals
x_sine    = build_song(As, keys, durs, fs);
x_violin  = build_song_violin(As, keys, durs, fs);
x_adsr    = build_song_adsr(As, keys, durs, ratios, fs);

% Duration of first four notes
T4 = sum(durs(1:4));           % seconds
N4 = floortol(T4 * fs);       % samples
t4 = (0:N4-1).' / fs;         % time axis (seconds)

figure;

% Sine
subplot(3,1,1)
plot(t4, x_sine(1:N4));
xlabel('Time (seconds)');
ylabel('Amplitude');
title('First Four Notes - Sine (build\_song)');

% Violin
subplot(3,1,2)
plot(t4, x_violin(1:N4));
xlabel('Time (seconds)');
ylabel('Amplitude');
title('First Four Notes - Violin (build\_song\_violin)');

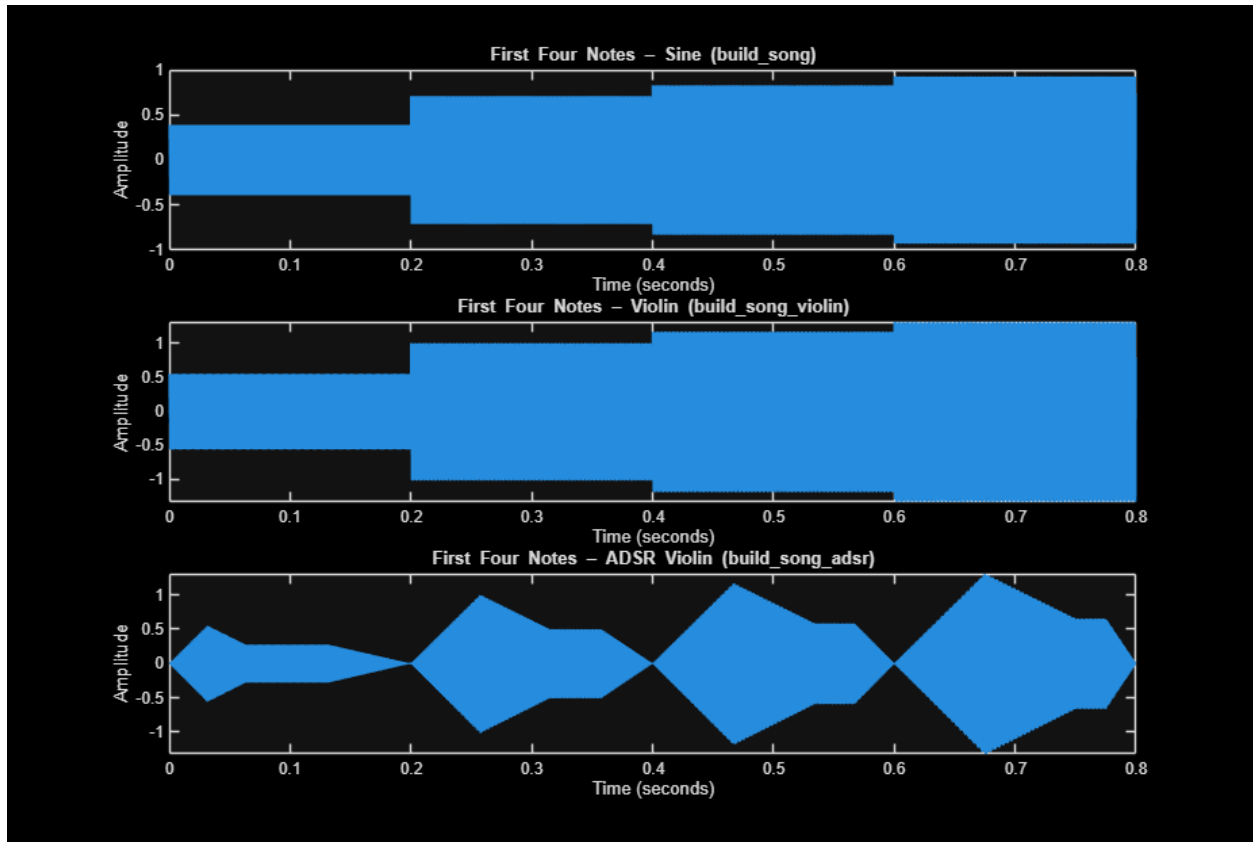
% ADSR Violin
subplot(3,1,3)

```

```

plot(t4, x_adsr(1:N4));
xlabel('Time (seconds)');
ylabel('Amplitude');
title('First Four Notes - ADSR Violin (build\_song\_adsr)');

```



=====

=====

4. AI for Audio Enhancement

=====

=====

```

% -----
% 4 (a)
% -----
%
% *** INCLUDE PROMPT HERE ***
%
% {

```

You are acting as a MATLAB signal-processing assistant.

Modify the `key_to_note_violin` MATLAB function shown below, which produces a single note for audio synthesis, representing a violin, from an amplitude "A", frequency key "key", duration "dur", and sampling frequency "fs". We aim to enhance this function to produce a more engaging synthetic sound. The signal must satisfy all of the following properties:

- The instrument should be a trumpet-like synthesizer.
- The timbral effect should be a bright, brassy sound with strong low harmonics and a slightly aggressive tone.
- The same inputs must produce an output "x" with the same fundamental frequency and duration as achieved by the correct function. The output "x" maximum and minimum amplitudes should always linearly scale with A.

Show your function here.

```

here is the key to note function by the way: function x =
key_to_note_violin(A, key, dur, fs)
% key_to_note_violin: Produces a sinusoidal waveform corresponding to a
% given piano key number
%
% Input Args:
% A: complex amplitude
% key: number of the note on piano keyboard
% dur: duration of each note (in seconds)
% fs: A scalar sampling rate value
%
% Output:
% x: sinusoidal waveform of the note
% # of samples and time vector
N = floortol(durfs);
t = (0:(N-1)).'/fs;
% Fundamental frequency from piano key number
f0 = (440/32)2^((key-9)/12);
% Violin harmonic table
Ak = [1.00, 0.35, 0.20, 0.12, 0.08, 0.04];
phik = [0.00, 1.20,-0.85, 0.65,-1.30, 0.45];
% Sum harmonics
y = zeros(N,1);
for k = 1:6
y = y + Ak(k)exp(1j(2pi(k*f0)*t + phik(k)));
end
% Apply overall complex amplitude and take real part
x = real(A * y);
end

%}

% -----
% 4 (b)
% -----
% Only need to modify function -- this area can be empty

% -----
% 4 (c)
% -----

```

```
music_ai = build_song_ai(As, keys, durs, fs);
audiowrite('music_ai.wav', music_ai/max(abs(music_ai)), fs);
```

```
===== SUPPORTING FUNCTIONS FOUND BELOW
=====
```

```
function x = key_to_note(A, key, dur, fs)
% key_to_note:
%
% Input Args:
%   A: complex amplitude
%   key: number of the note on piano keyboard
%   dur: duration of each note (in seconds)
%   fs: A scalar sampling rate value
%
% Output:
%   x: sinusoidal waveform of the note
```

```
    N = floortol(dur*fs);
    t = (0:(N-1)).'/fs;
    freq = (440/32)*2^((key-9)/12);
    x = real(A*exp(1j*2*pi*freq*t));
```

```
end
```

```
function x = build_song(As, keys, durs, fs)
% build_song:
%
% Input Args:
%   As: A length-N array of complex amplitudes for building notes
%   keys: A length-N array of key numbers (which key on a keyboard) for
building notes
%   durs: A length-N array of durations (in seconds) for building notes
%   fs: A scalar sampling rate value
%
% Output Args:
%   x: A length-(N*fs) length raw audio signal
%
```

```
    x = zeros(floortol(sum(durs)*fs), 1);
    for k = 1:length(keys)
        note = key_to_note(As(k), keys(k), durs(k), fs);
        start_time = sum(durs(1:k-1));
        n1 = floortol(start_time*fs) + 1;
        n2 = floortol(start_time*fs) + floortol(durs(k)*fs);
        x(n1:n2) = x(n1:n2) + note;
    end
```

```
end
```

```
end
```

```
function x = key_to_note_violin(A, key, dur, fs)
```

```

% key_to_note_violin: Produces a sinusoidal waveform corresponding to a
%   given piano key number
%
% Input Args:
%   A: complex amplitude
%   key: number of the note on piano keyboard
%   dur: duration of each note (in seconds)
%   fs: A scalar sampling rate value
%
% Output:
%   x: sinusoidal waveform of the note

```

```

% # of samples and time vector
N = floortol(dur*fs);
t = (0:(N-1)).'/fs;

% Fundamental frequency from piano key number
f0 = (440/32)*2^((key-9)/12);

% Violin harmonic table
Ak = [1.00, 0.35, 0.20, 0.12, 0.08, 0.04];
phik = [0.00, 1.20, -0.85, 0.65, -1.30, 0.45];

% Sum harmonics
y = zeros(N,1);
for k = 1:6
    y = y + Ak(k)*exp(1j*(2*pi*(k*f0)*t + phik(k)));
end

% Apply overall complex amplitude and take real part
x = real(A * y);

```

```
end
```

```

function x = build_song_violin(As, keys, durs, fs)
% build_song_violin:
%
% Input Args:
%   As: A length-N array of complex amplitudes for building notes
%   keys: A length-N array of key numbers (which key on a keyboard) for
building notes
%   durs: A length-N array of durations (in seconds) for building notes
%   fs: A scalar sampling rate value
%
% Output Args:
%   x: A length-(N*fs) length raw audio signal
%
% Pre-allocate full song length
x = zeros(ceil(sum(durs)*fs), 1);

for n = 1:length(keys)
    % Generate violin note and start time

```

```

    note = key_to_note_violin(As(n), keys(n), durs(n), fs);
    start_time = sum(durs(1:n-1));

    % Convert times to sample indices and add note
    n1 = floortol(start_time*fs) + 1;
    n2 = floortol(start_time*fs) + floortol(durs(n)*fs);
    x(n1:n2) = x(n1:n2) + note;
end

end

function x = key_to_note_adsr(A, key, dur, ratio, fs)
% key_to_note_adsr: Produces a sinusoidal waveform corresponding to a
%   given piano key number
%
% Input Args:
%   A: complex amplitude
%   key: number of the note on piano keyboard
%   dur: duration of each note (in seconds)
%   ratio: Ratio of (attack + decay) length to (sustain + release) length
%   fs: A scalar sampling rate value
%
% Output:
%   x: sinusoidal waveform of the note

% Force ratio into [0,1]
ratio = max(0, min(1, ratio));

% Samples and time vector
N = floortol(dur*fs);
t = (0:(N-1)).'/fs;

% Fundamental frequency
f0 = (440/32)*2^((key-9)/12);

% Violin harmonic table
Ak = [1.00, 0.35, 0.20, 0.12, 0.08, 0.04];
phik = [0.00, 1.20, -0.85, 0.65, -1.30, 0.45];

% Build violin waveform (unit-ish), then take real part
y = zeros(N,1);
for k = 1:6
    y = y + Ak(k)*exp(1j*(2*pi*(k*f0)*t + phik(k)));
end
y = real(y);

% ADSR segment lengths in samples
Na = floortol((ratio/2)*dur*fs);
Nd = floortol((ratio/2)*dur*fs);
Ns = floortol(((1-ratio)/2)*dur*fs);
Nr = floortol(((1-ratio)/2)*dur*fs);
%Nr = N - (Na + Nd + Ns); % leftover so total length = N

% Build envelope E(t): peak = A, sustain = A/2

```

```

E = zeros(N,1);
idx = 1;

if Na > 0
    E(idx:idx+Na-1) = linspace(0, A, Na);
    idx = idx + Na;
end
if Nd > 0
    E(idx:idx+Nd-1) = linspace(A, A/2, Nd);
    idx = idx + Nd;
end
if Ns > 0
    E(idx:idx+Ns-1) = (A/2)*ones(Ns,1);
    idx = idx + Ns;
end
if Nr > 0
    E(idx:idx+Nr-1) = linspace(A/2, 0, Nr);
end

% Apply ADSR envelope (Lab 0 style)
x = y .* E;

end

function x = build_song_adsr(As, keys, durs, ratios, fs)
% build_song:
%
% Input Args:
%     As: A length-N array of complex amplitudes for building notes
%     keys: A length-N array of key numbers (which key on a keyboard) for
building notes
%     durs: A length-N array of durations (in seconds) for building notes
%     ratios: A length-N array of ratios for each note
%     fs: A scalar sampling rate value
%
% Output Args:
%     x: A length-(N*fs) length raw audio signal
%
% Pre-allocate full song length
x = zeros(ceil(sum(durs)*fs), 1);

for n = 1:length(keys)

    % Make the n-th ADSR violin note
    note = key_to_note_adsr(As(n), keys(n), durs(n), ratios(n), fs);

    % Start time of this note (sum of previous durations)
    start_time = sum(durs(1:n-1));

    % Convert start time/duration to sample indices
    n1 = floortol(start_time*fs) + 1;
    n2 = n1 + length(note) - 1;

```

```

        % Add into the full song
        x(n1:n2) = x(n1:n2) + note;
    end
end

function x = key_to_note_ai(A, key, dur, fs)

% key_to_note_trumpet: Produces a bright, brassy trumpet-like waveform.
%
%   x = key_to_note_trumpet(A, key, dur, fs)
%
% INPUTS
%   A    - desired peak amplitude (scalar, real).
%          Output real part will lie in [-A, +A].
%   key  - integer piano key number (e.g. 60 = middle C).
%   dur  - duration of the note in seconds.
%   fs   - sampling frequency in Hz.
%
% OUTPUT
%   x    - time-domain waveform with the requested fundamental
%          frequency, duration, and a trumpet-like timbre.
%
% The function keeps the same fundamental frequency as the original
% key_to_note_violin implementation by using its MIDI-like frequency
% calculation. It synthesises a small number of harmonics that favour
% low-order odd harmonics (bright, brassy) while keeping higher
% harmonics weaker and applying a very short attack envelope to mimic
% the aggressive attack of a trumpet. The output is normalised so
% that its peak amplitude is exactly A, guaranteeing linear scaling
% with the requested amplitude.

```

1. Time vector

```

N = floor(dur*fs);           % number of samples (truncate last partial
sample)
t = (0:N-1).'/fs;           % column vector of time points

```

2. Fundamental frequency (same as key_to_note_violin)

```
f0 = (440/32)*2^((key-9)/12);
```

3. Harmonic synthesis

Number of harmonics we want to include

```
H = 8;
```

```

% Amplitude envelope for the harmonics. Strong first (fundamental),
% dropping roughly like 1/n for odd harmonics and 1/(2n) for even,
% then cutting the highest two to avoid excessive harshness.
amp = zeros(1,H);
for n = 1:H
    if mod(n,2) == 1
        amp(n) = 1/n;           % odd harmonics - bright
    else
        amp(n) = 1/(2*n);       % even harmonics - softer
    end
end
% Dampen the last two harmonics to keep the sound natural.
amp(end-1:end) = amp(end-1:end)*0.5;

% Phases that mimic the slightly aggressive quality of brass strings.
ph = [0, 0.2, -0.5, 0.7, -0.9, 0.4, -0.3, 0.6];

% Build the additive harmonic waveform
y = zeros(N,1);
for n = 1:H
    y = y + amp(n)*sin(2*pi*n*f0*t + ph(n));
end

```

4. Normalise to unit peak before scaling by A

```

maxy = max(abs(y));
if maxy > 0
    y = y / maxy;           % now max(abs(y)) == 1
end

```

5. Attack envelope (very short, 5 ms exponential)

```

tau = 0.005;               % 5ms time constant
e = 1 - exp(-t/tau);       % ramps from 0 to ~1
e(e>1) = 1;

```

6. Apply envelope, scale by A, and return real part

```

x = real(A * e .* y);

end

function x = build_song_ai(As, keys, durs, fs)
% build_song_violin:
%
% Input Args:
%   As: A length-N array of complex amplitudes for building notes
%   keys: A length-N array of key numbers (which key on a keyboard) for

```

```

building notes
%   durs: A length-N array of durations (in seconds) for building notes
%   fs: A scalar sampling rate value
%
% Output Args:
%   x: A length-(N*fs) length raw audio signal
%

% Pre-allocate full song length
x = zeros(ceil(sum(durs)*fs), 1);

for n = 1:length(keys)
    % Generate the n-th note
    note = key_to_note_ai(As(n), keys(n), durs(n), fs);

    % Compute start index for this note
    start_time = sum(durs(1:n-1));
    n1 = floortol(start_time*fs) + 1;
    n2 = n1 + length(note) - 1;

    % Add into the full song
    x(n1:n2) = x(n1:n2) + note;
end

end

function x = floortol(x)
%FLOORTOL Apply floor operation after adding 0.5 to ensure no
% floating-point rounding errors that unintendedly decrease the
% value
%

    x = floor(x+0.5);

end

```

Published with MATLAB® R2025b